Automated Warehouse Robot Controller

Complete Beginner's Guide

From Zero Knowledge to a Working Simulation

What This Guide Covers

- What the project is and **why** it exists in the real world
- Every AI concept explained from scratch, with analogies
- A step-by-step roadmap: what to do, in what order, and why
- The algorithms you must implement, explained simply
- How to build the simulation and what to submit

Project 9 — Multi-Agent Pathfinding (MAPF)
AI & Search Algorithms Course

Contents

1. Big Picture: What Is This Project About?

1.1. The Real-World Problem

Imagine you walk into an Amazon warehouse. It is a building the size of several football fields. Inside, there are thousands of shelves filled with products. When someone orders a book online, a **robot** needs to go find the shelf that holds that book and bring it to a human worker (called a "picker") who then packs it into a box.

Now here is the challenge: there are **hundreds of robots** all moving at the same time in the same space. How do they avoid crashing into each other? How do they not get stuck? How do you make sure all orders are packed as fast as possible?

This exact system is used by Amazon (they call it Kiva robotics, now Amazon Robotics). Your project asks you to **simulate and solve this problem** using Artificial Intelligence search algorithms.

1.2. Your Goal in One Sentence

Your goal: Build a software simulation where N robots move on a grid (the warehouse floor), each robot must go from its starting position to a target shelf, and **no two robots ever crash into each other**. You will use AI pathfinding algorithms to compute the paths.

1.3. Why Is This Hard?

Finding a path for **one** robot on a grid is easy — you probably already know the A* algorithm. The difficulty here is coordinating **multiple robots simultaneously**. Problems that arise:

- **Vertex conflict:** Two robots try to be in the same cell at the same time.
- **Edge conflict (head-on):** Robot A moves from cell X to cell Y, while Robot B moves from Y to X, in the same time step. They swap positions and crash.
- **Deadlock:** Four robots form a circular wait — each is blocked by the next, and nobody can move.
- **Congestion:** In a narrow corridor, robots queue up and waste time.

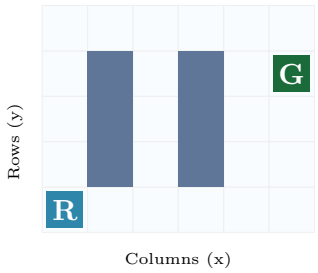
2. Background Knowledge You Need

2.1. What Is a Grid Map?

Think of the warehouse floor as a grid of squares, like graph paper. Each square is either:

- **Free (0):** The robot can stand here or move through it.
- **Obstacle (1):** A shelf or wall. The robot cannot enter this square.

A robot occupies exactly one cell at a time. At each time step, it can move to an adjacent cell (up, down, left, right) or stay in place (wait).



Blue squares = obstacles (shelves). R = Robot start. G = Goal.

2.2. What Is the A* Algorithm?

A* is the most important algorithm in this project. Before you tackle multi-robot problems, you must understand single-robot A*.

2.2.1. The Idea

A* finds the shortest path from a start cell to a goal cell on a grid. It is smarter than brute-force search because it uses a **heuristic** — a clever guess about how far you still are from the goal — to prioritize which cells to explore first.

2.2.2. The Formula

Every cell gets a score:

$$f(n) = g(n) + h(n)$$

- $g(n)$ = the number of steps it actually took to reach cell n from the start.
- $h(n)$ = a *heuristic estimate* of the remaining steps from n to the goal.
- $f(n)$ = the total estimated cost of the path through n.

For grids, the best heuristic is **Manhattan Distance**:

$$h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

This counts how many horizontal + vertical moves would be needed if there were no obstacles.

2.2.3. How A* Works (Step by Step)

1. Put the start cell in an **open list** (cells to be explored).
2. Pick the cell with the lowest f score from the open list.
3. If it is the goal — you are done! Trace back the path.
4. Otherwise, look at all its neighbors. For each neighbor:

- Calculate its g (parent's $g + 1$) and h (Manhattan distance to goal).
- If we have not visited it before, add it to the open list.

5. Move the current cell to a **closed list** (cells already processed).
6. Go back to step 2.

For a single robot, the state is just **(x, y)** — its position on the grid.
For multi-robot planning, we will extend the state to **(x, y, time)** — a 3-dimensional space-time grid. This is the key insight behind all multi-robot algorithms in this project.

2.3. What Is MAPF?

Multi-Agent Pathfinding (MAPF) is the formal name for the problem of finding collision-free paths for multiple agents simultaneously on a shared graph (your warehouse grid).

Formally: given N robots with start positions $s_1, s_2, \ldots, s_N$ and goal positions $g_1, g_2, \ldots, g_N$, find N paths (one per robot) such that:

1. Each path goes from the robot's start to its goal.
2. No two paths share the same cell at the same time step (no vertex conflict).
3. No two paths have robots swap positions in the same time step (no edge conflict).

3. Step-by-Step Project Roadmap

3.1. Overview of Phases

Phase	What You Do	Time
1	Set up the warehouse grid (the map)	Day 1
2	Implement single-robot A*	Day 2
3	Implement Independent A* (baseline)	Day 3
4	Implement Cooperative A* with reservation tables	Day 4–5
5	Build the GUI simulation	Day 6
6	Add heatmap and statistics	Day 7
7	Implement CBS (bonus)	Day 8
8	Run comparisons and write report	Day 9–10

Table 1: Project phases and suggested timeline

3.2. Phase 1 — Build the Warehouse Grid

3.2.1. What to Do

Create a 2D array (a list of lists in Python) where each cell is either 0 (free) or 1 (wall/obstacle). A typical warehouse has:

- Outer walls on all four borders.
- Shelf rows running vertically or horizontally, leaving corridors in between.
- Open areas near the "packing stations" (where robots deliver shelves).

3.2.2. Why

This is the foundation of everything. Every algorithm you write will read this grid to know where robots can and cannot go. Without a proper map, nothing works.

3.2.3. Example Grid in Python

Listing 1: Simple warehouse grid construction

```
GRID_WIDTH  = 20
GRID_HEIGHT = 20

def build_warehouse():
    grid = []
    for y in range(GRID_HEIGHT):
        row = []
        for x in range(GRID_WIDTH):
            # Outer walls
            if x == 0 or y == 0 or x == GRID_WIDTH-1 or y ==
               GRID_HEIGHT-1:
                row.append(1)
            # Shelf columns every 4 cells, leaving corridors
            elif x % 4 == 2 and y > 1 and y < GRID_HEIGHT - 2
               and y % 2 == 0:
                row.append(1)
            else:
                row.append(0)
        grid.append(row)
    return grid
```

3.2.4. Deliverable

A printed or drawn grid showing obstacles and free space. If you visit a local warehouse (bonus), sketch their floor plan instead.

3.3. Phase 2 — Implement Single-Robot A*

3.3.1. What to Do

Write a function `astar(grid, start, goal)` that returns the shortest path from `start` to `goal` as a list of (x, y) positions.

3.3.2. Why

Every multi-robot algorithm is built on top of single-robot A*. You must get this right first. Think of it as the "engine" that all other algorithms will use.

3.3.3. Data Structures You Need

- **Open list:** A priority queue (use Python's `heapq`), sorted by f score.
- **Closed set:** A set of already-processed states.
- **Came-from dictionary:** Maps each state to its parent, so you can reconstruct the path.

Listing 2: Single-robot A* implementation

```
import heapq

def heuristic(a, b):
    """Manhattan distance heuristic."""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar_single(grid, start, goal):
    """
    Find shortest path from start to goal on grid.
    start, goal: (x, y) tuples
    Returns: list of (x, y) positions, or empty list if no path
    """
    open_list = []
    heapq.heappush(open_list, (0, start))

    came_from = {}
    g_score   = {start: 0}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            # Reconstruct path
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return list(reversed(path))

        x, y = current
        for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < len(grid[0]) and 0 <= ny < len(grid):
                if grid[ny][nx] == 0:   # not a wall
                    neighbor    = (nx, ny)
                    tentative_g = g_score[current] + 1
```

```
                    if tentative_g < g_score.get(neighbor,
                       float('inf')):
                        came_from[neighbor] = current
                        g_score[neighbor]   = tentative_g
                        f = tentative_g + heuristic(neighbor,
                           goal)
                        heapq.heappush(open_list, (f, neighbor)
                           )

    return []  # No path found
```

3.3.4. Test It

Before continuing, test your A* on a small 5×5 grid manually and check the output path makes sense.

3.4. Phase 3 — Independent A* (Baseline)

3.4.1. What to Do

Plan a path for every robot **independently** as if the other robots do not exist. Then simulate all robots moving simultaneously along their planned paths, and check whether collisions happen.

3.4.2. Why

This is your **baseline** — the simplest possible approach. It will fail (cause collisions) in crowded warehouses. You need this to demonstrate *why* smarter algorithms are necessary, and to compare against later.

Listing 3: Independent A* planning

```
def plan_independent(grid, starts, goals):
    """
    Each robot plans independently, ignoring others.
    Returns: list of paths (one per robot).
    """
    paths = []
    for i in range(len(starts)):
        path = astar_single(grid, starts[i], goals[i])
        paths.append(path)
    return paths

def count_conflicts(paths):
    """Check how many vertex/edge conflicts exist in a set of
       paths."""
    max_len = max(len(p) for p in paths)
    conflicts = 0
    for t in range(max_len):
        positions = []
        for path in paths:
            pos = path[min(t, len(path)-1)]
```

```
            positions.append(pos)
        # Vertex conflict: two robots at same cell
        if len(positions) != len(set(positions)):
            conflicts += 1
    return conflicts
```

3.4.3. Expected Result

With many robots and a dense map, `count_conflicts()` will return a positive number. This demonstrates the problem that the next algorithms solve.

3.5. Phase 4 — Cooperative A* with Reservation Tables

3.5.1. What to Do

This is the **main algorithm** of the project. Plan robots one at a time (robot 1 first, then robot 2, etc.). When planning robot k, treat all previously planned robots' paths as "reserved" space-time slots that robot k cannot enter.

3.5.2. The Key Concept: Reservation Table

A **reservation table** (also called a space-time grid) is a data structure that records which cells are occupied at which time steps.

Instead of searching in 2D space (x, y), we search in 3D space-time (x, y, t).
A state is valid only if no other robot has reserved the cell (x, y) at time t.
This extends A* naturally: the state space is larger, but the algorithm stays the same.

3.5.3. Why This Works

When robot 1 plans its path, it uses cells $c_1, c_2, c_3, \ldots$ at times $t = 0, 1, 2, \ldots$. We record all of these as "reserved." When robot 2 runs A*, its state (x, y, t) is blocked if (x, y, t) is in the reservation table. This guarantees robot 2 never collides with robot 1. We repeat for robots 3, 4, 5, etc.

Listing 4: Cooperative A* with reservation table

```
def astar_spacetime(grid, start, goal, reserved):
    """
    Space-Time A*: searches in (x, y, time) space.
    reserved: a set of (x, y, t) tuples that are forbidden.
    Returns: list of (x, y) positions.
    """
    # State is (x, y, time)
    start_state = (start[0], start[1], 0)
    open_list   = []
    heapq.heappush(open_list, (0, start_state))

```

```
    came_from = {}
    g_score   = {start_state: 0}
    MAX_T     = len(grid) * len(grid[0]) * 2  # safety limit

    while open_list:
        _, current = heapq.heappop(open_list)
        cx, cy, ct = current

        if (cx, cy) == goal:
            # Reconstruct spatial path
            path = []
            while current in came_from:
                path.append((current[0], current[1]))
                current = came_from[current]
            path.append(start)
            return list(reversed(path))

        if ct > MAX_T:
            continue

        # Moves: wait + 4 directions
        for dx, dy in [(0,0),(1,0),(-1,0),(0,1),(0,-1)]:
            nx, ny, nt = cx+dx, cy+dy, ct+1
            if not (0 <= nx < len(grid[0]) and 0 <= ny < len(
               grid)):
                continue
            if grid[ny][nx] == 1:           # wall
                continue
            if (nx, ny, nt) in reserved:    # reserved by
               another robot
                continue
            # Check edge conflict (swap)
            if (nx, ny, ct) in reserved and (cx, cy, nt) in
               reserved:
                continue

            neighbor      = (nx, ny, nt)
            tentative_g   = g_score[current] + 1
            if tentative_g < g_score.get(neighbor, float('inf')
               ):
                came_from[neighbor] = current
                g_score[neighbor]   = tentative_g
                h = abs(nx - goal[0]) + abs(ny - goal[1])
                heapq.heappush(open_list, (tentative_g + h,
                   neighbor))

    return [start]  # fallback: stay in place


def plan_cooperative(grid, starts, goals):
    """
```

```
    Plan paths sequentially. Each robot avoids all previously
       planned robots.
    """
    reserved = set()
    paths    = []

    for i in range(len(starts)):
        path = astar_spacetime(grid, starts[i], goals[i],
           reserved)
        paths.append(path)

        # Reserve every (x, y, t) that robot i will occupy
        for t, pos in enumerate(path):
            reserved.add((pos[0], pos[1], t))

        # Also reserve the goal position for future times
        # (robot stays at goal after arriving)
        goal_pos = path[-1]
        for t in range(len(path), len(path) + 20):
            reserved.add((goal_pos[0], goal_pos[1], t))

    return paths
```

3.5.4. Why the Order of Planning Matters

The first robot planned has full freedom. The last robot planned has the most constraints. In practice, you can order robots by distance to goal (shortest paths first) to reduce unnecessary waiting. This is an optional optimization.

3.6. Phase 5 — Hill Climbing Optimization

3.6.1. What to Do

After Cooperative A* gives you a valid (collision-free) set of paths, try to shorten them using Hill Climbing. This is an **optimization phase**, not a planning phase.

3.6.2. Simple Version

For each robot, check if there is a waiting step (robot stays in the same cell for two consecutive steps). Try removing it. If the resulting path is still collision-free, keep the shorter path; otherwise discard the change.

Listing 5: Simple Hill Climbing optimization

```
def remove_waits(paths, grid):
    """
    Try to remove unnecessary wait steps from each robot's path
       .
    Returns improved paths.
    """
    improved = True
    while improved:
```

```
        improved = False
        for i, path in enumerate(paths):
            new_path = []
            j = 0
            while j < len(path):
                new_path.append(path[j])
                # Skip duplicate consecutive positions (waits)
                while j + 1 < len(path) and path[j] == path[j
                    +1]:
                    j += 1
                j += 1
            if len(new_path) < len(path):
                # Verify no new conflicts introduced
                test_paths    = paths[:i] + [new_path] + paths[
                    i+1:]
                if count_conflicts(test_paths) == 0:
                    paths[i]  = new_path
                    improved  = True
    return paths
```

3.7. Phase 6 — CBS: Conflict-Based Search (Bonus)

3.7.1. What to Do

CBS is a more advanced algorithm that finds **optimal** conflict-free paths (minimum total cost). It uses a two-level search:

- **Low level:** A standard single-robot A* (or space-time A*) for one agent with extra constraints.
- **High level:** A search over a "Conflict Tree" where each node represents a set of constraints.

3.7.2. How the Conflict Tree Works

1. Start with the root node: plan each robot independently (no constraints).
2. Find the **first conflict** (earliest time step where two robots collide).
3. Create two child nodes:
    - Child A: add a constraint "Robot i cannot be at cell (x, y) at time t", then replan robot i.
    - Child B: add a constraint "Robot j cannot be at cell (x, y) at time t", then replan robot j.
4. Add both children to the open list. Pick the one with the lowest total path cost.
5. Repeat until you find a node with **no conflicts**.

Cooperative A* is a greedy approach — it gives priority to robots planned first. The last robot may take a very long detour. CBS tries all possible constraint assignments and finds the globally optimal solution, but may explore many more nodes. It is better for correctness; Cooperative A* is better for speed.

3.8. Phase 7 — Build the GUI Simulation

3.8.1. What to Do

Use Python's `pygame` or `tkinter` library (or a web-based canvas as provided in this project) to draw the warehouse grid and animate robots moving along their computed paths.

3.8.2. Minimum Requirements

- Robots displayed as colored circles, each with a unique ID.
- Goals displayed as matching-color rings or markers.
- Walls/shelves displayed in a contrasting color.
- Animation: step through time, moving each robot one cell per step.
- Collision detection: highlight robots in red when a conflict occurs.
- Display statistics: current time step, conflicts detected, robots done.

3.8.3. Optional (Highly Recommended)

- A **heatmap** overlay showing which corridors are used most (see Phase 8).
- Buttons to switch algorithms and compare results side-by-side.
- A counter for Makespan and Flowtime.

3.9. Phase 8 — Heatmap and Performance Statistics

3.9.1. What to Do

Track how many times each cell is visited by any robot during the simulation. Normalize these counts and display them as a color gradient: cold colors for rarely visited cells, hot colors for heavily trafficked corridors.

3.9.2. Why

The heatmap directly answers the question: "Which corridors are bottlenecks?" In a real warehouse, this information is used to redesign shelf layouts to reduce congestion.

Listing 6: Computing the heatmap

```
1 def compute_heatmap(paths, grid_height, grid_width):
```

```
    """Count how often each cell is visited across all robots
       and all time steps."""
    heat = [[0]*grid_width for _ in range(grid_height)]
    for path in paths:
        for (x, y) in path:
            heat[y][x] += 1
    return heat
```

4. The Two Key Metrics You Must Measure

4.1. Makespan

$$\text{Makespan} = \max_{i=1}^{N} \text{len}(\text{path}_i) - 1$$

This is the time (in steps) until the **last** robot reaches its goal. Think of it as "how long does the warehouse stay busy?"

4.2. Flowtime

$$\text{Flowtime} = \sum_{i=1}^{N} \big(\text{len}(\text{path}_i) - 1\big)$$

This is the **sum** of all individual completion times. Think of it as "how much total waiting/moving did all robots do?"

Metric	What it measures	When it matters
Makespan	Time until the slowest robot finishes	When you care about throughput
Flowtime	Total effort by all robots combined	When you care about energy/cost

Table 2: Comparison of the two objective metrics

A good solution minimizes both. Sometimes they conflict: a path that helps one robot finish faster forces another to wait longer.

5. Conflict Types and How to Detect Them

5.1. Vertex Conflict

Definition: Two robots i and j are at the same cell (x, y) at the same time t.

$$\text{path}_i[t] = \text{path}_j[t] = (x, y)$$

Detection code:

```
# At time t, check all pairs of robots
positions_at_t = [path[min(t, len(path)-1)] for path in paths]
for i in range(len(positions_at_t)):
    for j in range(i+1, len(positions_at_t)):
        if positions_at_t[i] == positions_at_t[j]:
            print(f"Vertex conflict at t={t}: robots {i} and {j
                }")
```

5.2. Edge Conflict (Swap Conflict)

Definition: Robot i moves from cell u to cell v while robot j moves from v to u in the same time step $t \to t+1$.

$$\text{path}_i[t] = \text{path}_j[t+1] \quad \text{AND} \quad \text{path}_i[t+1] = \text{path}_j[t]$$

Detection code:

```
for t in range(max_t - 1):
    for i in range(len(paths)):
        for j in range(i+1, len(paths)):
            pi_now  = paths[i][min(t,   len(paths[i])-1)]
            pi_next = paths[i][min(t+1, len(paths[i])-1)]
            pj_now  = paths[j][min(t,   len(paths[j])-1)]
            pj_next = paths[j][min(t+1, len(paths[j])-1)]
            if pi_now == pj_next and pi_next == pj_now:
                print(f"Edge conflict at t={t}: robots {i} and
                    {j}")
```

5.3. Deadlock

A deadlock occurs when a group of robots form a cycle: Robot A waits for B's cell, B waits for C's cell, C waits for A's cell. None of them can move.

Prevention: Cooperative A* prevents deadlocks because each robot's path is fully planned (not just reactive). A robot always has a complete path to follow, so it never enters a cell without a planned exit.

6. Comparison Experiments You Must Run

6.1. Experiment 1: Conflict Rate

- Run Independent A* with 5, 10, and 15 robots on the same map.
- Count the number of conflicts in each case.
- Run Cooperative A* with the same settings.

- Expected result: Cooperative A* always yields 0 conflicts. Independent A* conflict rate increases with robot count.

6.2. Experiment 2: Makespan vs. Fleet Size

- Run Cooperative A* with 5, 10, 15, and 20 robots.
- Record Makespan and Flowtime for each.
- Create a bar chart or table showing how performance degrades as the warehouse gets more crowded.

6.3. Experiment 3: Head-On Collision Scenario

- Place Robot 1 at $(2, 10)$ with goal $(17, 10)$.
- Place Robot 2 at $(17, 10)$ with goal $(2, 10)$.
- They are on a direct collision course.
- Run Independent A*: observe the edge conflict.
- Run Cooperative A*: observe how Robot 2 plans an alternative route or waits.

Algorithm	**5 Robots**	**10 Robots**	**15 Robots**	
Independent A* (conflicts)	?	?	?	
Cooperative A* (conflicts)	0	0	0	
Independent A* (makespan)	?	?	?	
Cooperative A* (makespan)	?	?	?	

Table 3: Template for your experiment results table (fill in during testing)

7. Project Deliverables Checklist

✓ **Working code:** Python files implementing all algorithms.
✓ **Grid map:** Either a custom layout (from a local warehouse visit) or a MovingAI benchmark map.
✓ **GUI simulation:** Animated grid showing robots moving. Minimum 10 robots simultaneously.
✓ **Heatmap:** Color-coded congestion map after a simulation run.
✓ **Statistics table:** Makespan and Flowtime for each algorithm and fleet size.
✓ **Head-on demo:** A screenshot or recording showing the head-on scenario being resolved.
✓ **Documentation:** Explanation of the collision-checking logic in the time dimension.

7.1. Documentation: Collision Checking in the Time Dimension

This is specifically asked for in the project brief. Here is what to write:

In standard 2D A*, we check whether a cell (x, y) is a wall. In space-time A*, we check whether the **space-time cell** (x, y, t) is either a wall or reserved by another robot. The reservation table is a hash set (Python `set`) of tuples (x, y, t). Lookup in a hash set is $O(1)$, so checking a reservation costs nothing. When robot k finishes planning its path $p_k = [(x_0, y_0), (x_1, y_1), \ldots, (x_T, y_T)]$, we insert all tuples (x_t, y_t, t) for $t = 0, 1, \ldots, T$ into the reservation table. We also insert the goal position for several extra time steps, because the robot stays at its goal after arriving and must not be displaced by later robots.

Edge conflicts (swaps) are caught by checking: if robot k wants to move from (x, y) at time t to (x', y') at time $t+1$, it first checks whether (x', y', t) is reserved **and** $(x, y, t+1)$ is reserved — this would mean another robot is doing the exact reverse move.

8. Glossary of Terms

A* Pathfinding algorithm using $f = g + h$. Optimal and complete when heuristic is admissible.

MAPF Multi-Agent Pathfinding. The general problem of coordinating multiple agents on a graph.

Makespan The time step when the last robot finishes its task. Measures throughput.

Flowtime Sum of all individual completion times. Measures total system effort.

Reservation Table
A set of (x, y, t) tuples marking cells occupied by already-planned robots.

Space-Time A*
A* extended into 3D space-time; the state is (x, y, t).

Vertex Conflict
Two robots at the same cell at the same time.

Edge Conflict
Two robots swapping positions in the same time step.

Deadlock A cyclic wait where no robot can move.

CBS Conflict-Based Search. Optimal MAPF algorithm using a two-level conflict tree.

Heuristic An estimate of remaining cost. Must be admissible (never overestimate) for A* to be optimal.

Manhattan Distance
$|x_1 - x_2| + |y_1 - y_2|$. The standard admissible heuristic for grid maps.

Cooperative A*
Plan robots sequentially; each robot treats previous robots' paths as

dynamic obstacles.

Independent A*
: Plan each robot ignoring others; collisions handled at runtime (baseline, expected to fail).

Hill Climbing
: Local search optimization that iteratively improves a solution by small changes.

Packing Station
: The station where a robot delivers a shelf to a human worker for order fulfillment.

9. Recommended File Structure

Listing 7: Suggested project folder layout

```
warehouse_project/
|-- grid/
|   |-- warehouse_map.py        # Grid definition and builder
|   |-- maps/                   # Additional map files
|
|-- algorithms/
|   |-- astar.py                # Single-robot A*
|   |-- independent.py          # Independent A* planner
|   |-- cooperative.py          # Cooperative A* + reservation
    table
|   |-- cbs.py                  # Conflict-Based Search (bonus)
|   |-- hill_climbing.py        # Path optimization
|
|-- simulation/
|   |-- gui.py                  # pygame/tkinter visualization
|   |-- heatmap.py              # Congestion heatmap
|   |-- stats.py                # Makespan, Flowtime, conflict
    counter
|
|-- experiments/
|   |-- compare_algorithms.py   # Run experiments and collect
    data
|   |-- results/                # CSV or JSON output files
|
|-- main.py                     # Entry point: run the
    simulation
|-- report.pdf                  # Your documentation
```

10. Quick Reference Summary

Algorithm	How it works	Conflict-free?	Optimal?
Independent A*	Plan each robot alone; collisions happen live	No	Per-robot yes
Cooperative A*	Plan sequentially; use reservation table	Yes	No (greedy)
CBS	Two-level conflict tree search	Yes	Yes
Hill Climbing	Post-process: remove waits from valid paths	Yes (preserves)	Locally

Table 4: Summary of all algorithms

- Forgetting to reserve the **goal position after arrival** — late robots will try to walk through parked robots.
- Only checking vertex conflicts and ignoring **edge (swap) conflicts**.
- Using Euclidean distance as the heuristic — it is not admissible on grid maps with only 4-directional movement (use Manhattan distance).
- Running the simulation without a **maximum time limit** in A* — if no path exists, the search will run forever.
- Comparing algorithms on different random seeds — always use the **same start/-goal positions** for fair comparison.

Good luck!